



**University
of Victoria**

Final Project Documentation

MECH 458 - Mechatronics

Spring 2025

Group 8

Luca Carinelli

Olivia Daugela

Abstract

This project involved designing and building an automated object sorting system using an ATmega2560 microcontroller. The system uses a reflective sensor for material classification and a stepper motor to direct objects into the correct bin. It operates using interrupt-driven control and a FIFO queue to maintain sorting order. The final system successfully sorted 48 objects in 29 seconds with zero errors, with minimal spacing between items. Performance was improved through optimized belt speed and carefully timed stepper control. An S-curve acceleration profile was implemented for the stepper motor to achieve smooth and efficient bin rotation, contributing to the overall speed and reliability of the system. The project provided hands-on experience in real-time control, embedded programming, and hardware integration.

Acknowledgement

We would like to sincerely thank Dr. Yang Shi for his leadership and instruction throughout the MECH 458 course. His lectures provided us with a strong foundation in mechatronics, which was essential for understanding the core concepts behind embedded systems, sensor integration, and actuator control. The theoretical knowledge gained in class guided many of our design decisions and troubleshooting strategies during the project.

We would also like to extend our deepest appreciation to Mr. Patrick Chang for his exceptional support and dedication in the laboratory. His hands-on guidance was crucial in helping us overcome challenges related to coding, hardware integration, and microcontroller setup. Whether it was debugging complex issues or assisting with the physical assembly of our system, Mr. Chang's expertise and patience greatly contributed to the success of our final design.

Table of Contents

1.0 Introduction.....	4
1.1 Statement of the Problem.....	4
1.2 Purpose of the Design and Design Overview.....	5
2.0 Methods and Design Approach.....	5
2.1 Explanation and Description of the Flowchart.....	5
3.0 Results.....	5
3.1 Technical Description of the ATmega 2560.....	6
3.2 System Performance Specifications.....	6
3.3 System Algorithm.....	6
3.4 Operation.....	6
3.5 Testing and Calibration.....	6
3.6 Limitations and System Tradeoffs.....	7
4.0 Experience and Recommendations.....	7
5.0 Conclusion.....	7
References.....	8
Appendix A - Original Statement of Preliminary System Design.....	8
Appendix B - Detailed Technical Documentation.....	8

List of Figures

Figure 3: Flowchart of main project code.....	7
Figure 4: Flowchart of interrupts.....	7
Figure 5: Flowchart of ADC Interrupt.....	9
Figure B.1: Circuit diagram.....	38
Figure B.2: Block Diagram.....	38

1.0 Introduction

1.1 Statement of the Problem

This project aims to develop an automated sorting system that accurately classifies objects based on their material. A DC motor drives a conveyor belt, moving objects through a reflective sensor, which identifies their colour when triggered by an optical sensor. At the end of the belt, a second optical sensor detects the object's position, prompting a stepper motor to adjust the sorting tray and direct the object into the appropriate bin [1].

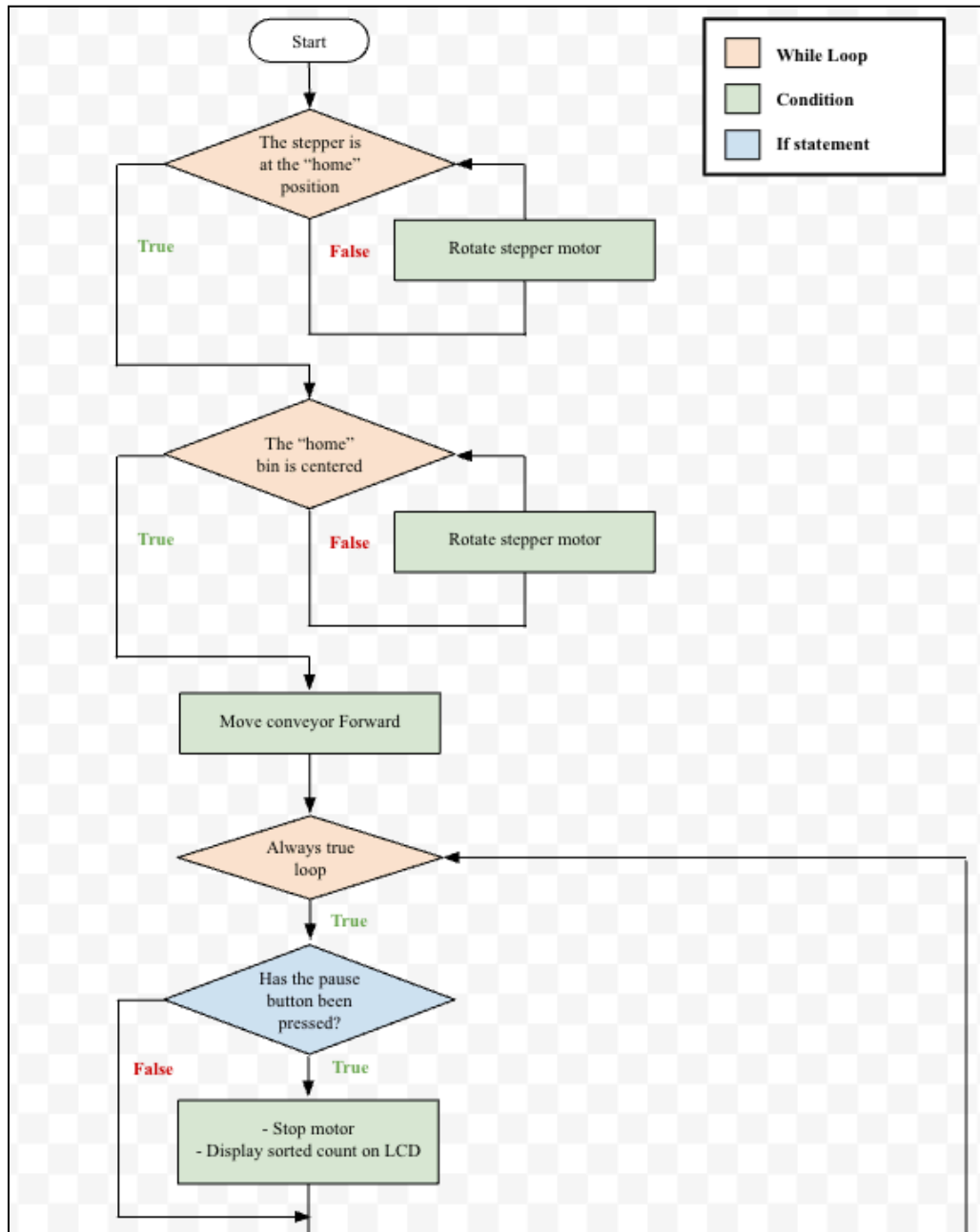
The system should sort 48 materials of varying types and colours in under 60 seconds with minimal errors while ensuring the safety of the machine and its users.

1.2 Purpose of the Design and Design Overview

The purpose of the design was to create a reliable, microcontroller-based system capable of accurately sorting objects in real time. The system was built around the ATmega2560, which handled all sensing, classification, and actuation tasks. Interrupts were used for responsive sensor detection, while ADC readings from the reflective sensor were used to classify materials. A FIFO queue maintained the correct sorting order as objects moved down the conveyor.

To ensure consistent performance, the system was designed with clear separation between sensing, processing, and control functions. Stepper motor rotation was timed precisely based on queue data, and the belt motor was managed using PWM signals. The overall design prioritized responsiveness, accuracy, and smooth coordination between hardware and software.

2.0 Methods and Design Approach



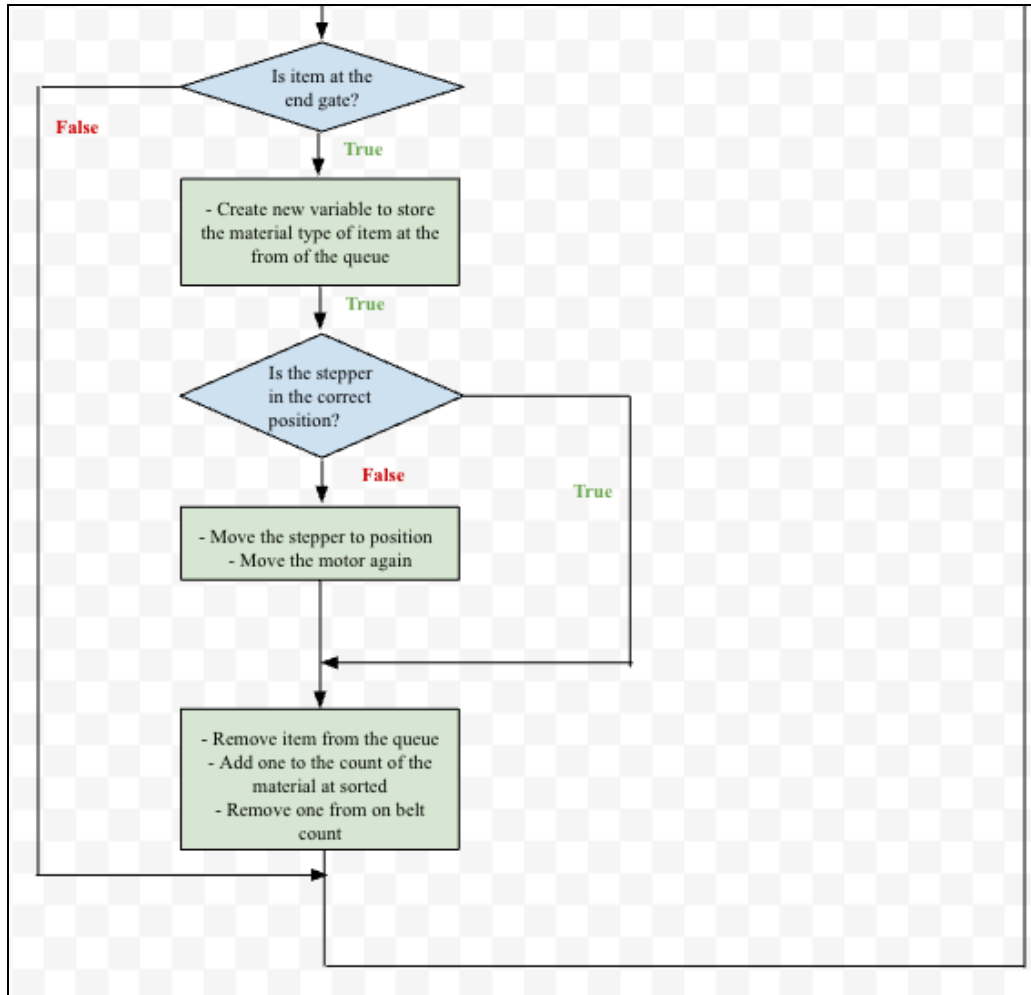


Figure 3: Flowchart of main project code

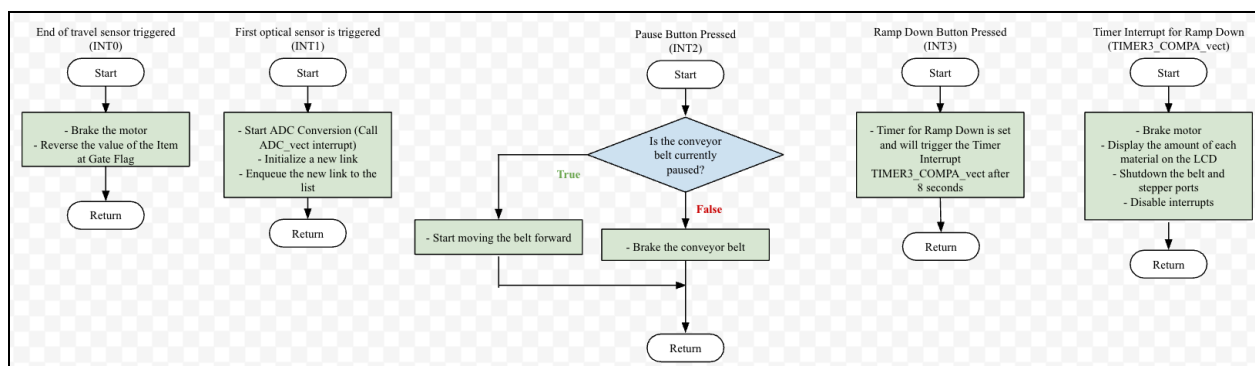
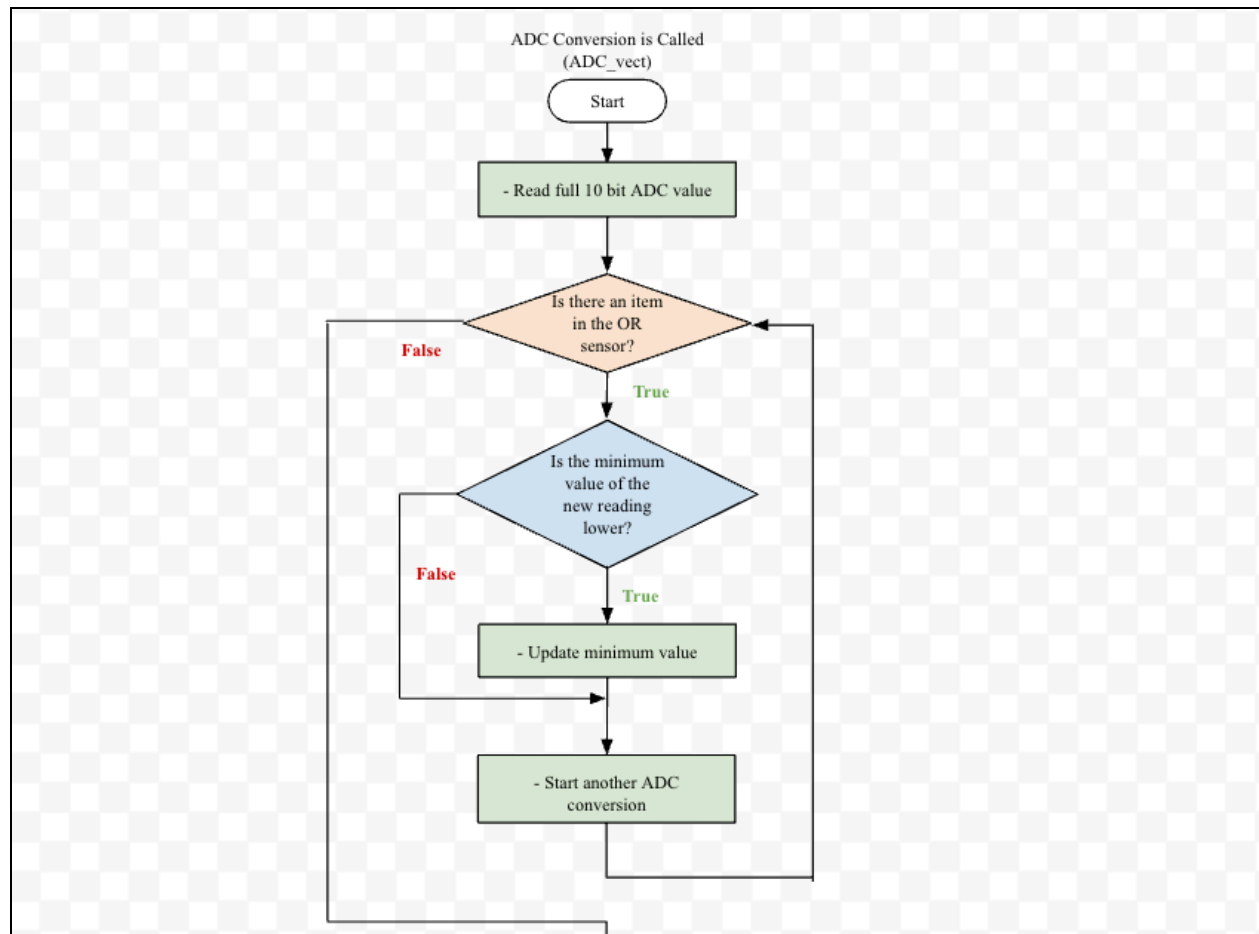


Figure 4: Flowchart of interrupts



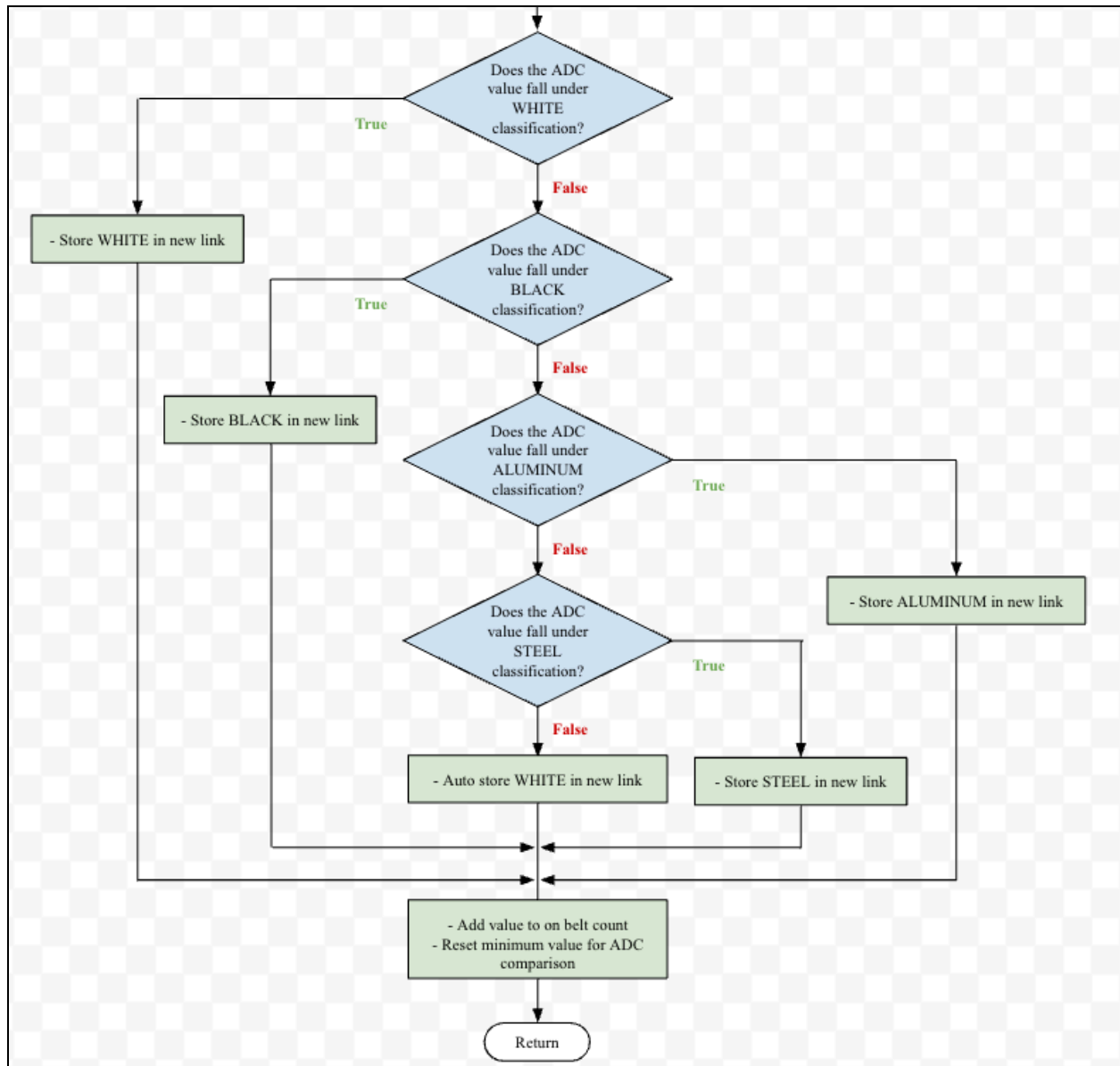


Figure 5: Flowchart of ADC Interrupt

2.1 Explanation and Description of the Flowcharts

The main program flow begins with an initialization sequence that ensures the stepper motor is in the home position and correctly centred. Once confirmed, the conveyor belt begins moving forward and enters a continuous loop that checks system inputs and handles sorting events.

An interrupt-driven structure allows the system to respond immediately to external events. When an object is detected by the first optical sensor, an ADC conversion is triggered via the INT1 interrupt. During this process, the system continuously captures ADC readings while the object is present and records the lowest value. After the object passes, this value is compared to predefined thresholds to classify the object as black, white, aluminum, or steel. A corresponding link is added to the classification queue.

As the object travels down the conveyor, the system checks if it reaches the end gate via the INT0 interrupt. At this point, the material type at the front of the queue is retrieved. If the sorting bin is already aligned with the expected material, the conveyor continues. If not, the stepper motor rotates to the appropriate position before allowing the object to be released.

User inputs are handled via INT2 and INT3. The pause button toggles the conveyor state and displays the count of sorted items on the LCD. The ramp-down button starts a timer interrupt (TIMER3_COMPA_vect) that fires approximately 3.9 seconds after activation, allowing the system to finish sorting current objects before halting all movement, updating the LCD, and disabling interrupts.

The system uses a mix of polling and interrupt routines to manage state updates, ensuring responsiveness without compromising sorting accuracy. This hybrid control structure keeps the belt running efficiently while handling dynamic events such as classification, bin rotation, and user interaction.

3.0 Results

3.1 Technical Description of the ATmega 2560

The system uses the ATmega2560 [11], an 8-bit AVR microcontroller with a 16 MHz clock speed, 256KB of flash memory, 8KB of SRAM, and 4KB of EEPROM. It includes 86 programmable I/O pins, a 16-channel 10-bit ADC, multiple timers, and PWM channels to control motors, process sensor data, and update the LCD.

The microcontroller has built-in timers that can be configured to generate precise delays for various system functions. In this system, the timer is set up to increment every 1 millisecond, ensuring accurate timing for stepper motor control, user input handling, and sensor processing.

The microcontroller also has a built-in pulse-width modulation (PWM) feature, which controls the speed of the conveyor motor. PWM works by modulating the power supplied to the motor in fast, repeated cycles. Adjusting the duty cycle (the percentage of time the signal is on), the system can regulate the motor speed as needed.

The microcontroller has built-in interrupts, which allow it to respond instantly to external signals. Interrupts are triggered by changes in sensor inputs or button presses, temporarily pausing the main program to handle critical tasks. The ATmega2560 supports 6 external interrupt inputs, with lower-numbered interrupts having higher priority, ensuring critical events are processed first.

The microcontroller has a built-in analog-to-digital converter (ADC) that converts sensor voltage signals into digital values. It operates at 10-bit resolution and produces values from 0 to 1023, where 0 corresponds to 0V and 1023 corresponds to 5V. The ADC result is stored in two 8-bit registers: ADCL and ADCH. Since the ADC result spans 10 bits, the lower 8 bits are read from ADCL, while the upper 2

bits are stored in ADCH and shifted left by 8 bits before being combined. This ensures the final ADC value is correctly reconstructed for processing.

3.2 System Performance Specifications

The final system was able to complete a full sort of 48 objects in 29 seconds, well within the 60-second target. Objects were placed at the start of the belt with minimal spacing, ensuring continuous detection and processing. The conveyor belt was driven by a DC motor operating at a PWM duty cycle of 50.78%. This speed provided a good balance between throughput and control—any faster, and objects would begin to slip off the end of the belt during braking, resulting in mis-sorts.

The stepper motor was controlled using an S-curve motion profile to improve stability and precision during rotation. The step delay ranged from 20 ms down to 6 ms, where 6 ms was the lowest stable delay without introducing missed steps or glitching. Additionally, a pre-release strategy was implemented: the object would be released while the stepper was still 2 steps away from its final position. This timing allowed the object to begin falling just before the bin was fully aligned, improving accuracy. A short settling delay of 80 milliseconds was also added after each bin rotation to ensure the object had time to drop cleanly into the bin before the system resumed operation.

To ensure accurate material classification, the ADC prescaler was set to 32x, reducing the clock from 8 MHz to 250 kHz. This slower conversion speed increased the reliability of readings, especially given the limited spacing between stacked objects and the need for real-time ADC comparisons during each pass.

Overall, the system was optimized for speed, precision, and repeatability. The final configuration achieved accurate classification and sorting under tight spacing and timing conditions, with minimal observed misreads or stepper faults during test runs.

3.3 System Algorithm

The sorting system is built around a main polling loop with multiple interrupt service routines (ISRs) to enable real-time responsiveness. The microcontroller continuously runs a loop that checks system states, but events such as object detection, classification, and user input are handled through interrupts.

Material classification begins when the first optical sensor triggers an external interrupt (INT1). This starts an ADC sampling routine that tracks the lowest reflectivity value while the object is within range. Once the object has passed, this value is used to classify the material, and a corresponding link is added to a FIFO queue. This queue makes sure objects are sorted in the order they were detected.

When the second optical sensor at the end of the conveyor triggers (INT0), the system reads the front of the queue to determine the required bin position. If the stepper is already aligned, the object is released immediately. If not, the stepper rotates using an S-curve motion profile. A pre-release mechanism begins moving the belt when the stepper is two steps away from its final position, improving accuracy. A short delay follows to ensure the object has time to fall cleanly into the bin.

User inputs are handled by two additional interrupts: the pause button toggles the belt state and displays the current object count, while the ramp-down button initiates a timed sequence that allows the system to finish sorting remaining objects before stopping all motion.

3.4 Operation

At startup, the system homes the stepper motor using the Hall Effect sensor and centers the bin. The conveyor belt then begins moving forward. As objects pass the first sensor, they are classified based on reflectivity and added to a queue. When they reach the end sensor, the system checks the bin position and rotates the stepper if needed. A pre-release and short delay ensure the object falls accurately into the correct bin. The LCD remains off during normal operation. It only updates when the pause or ramp-down button is pressed, displaying how many objects are currently on the belt and how many of each type have been sorted. This process continues until the ramp-down sequence completes and the system stops automatically.

3.5 Testing and Calibration

For the system to accurately classify materials, it was necessary to determine the range of ADC values each material produced. A separate calibration program was developed to establish these ranges. All parts of a single material were sent through the first optical sensor, triggering ADC conversions (as in the main code), with the system recording the lowest value while each object was present. After each part left the sensor, its value was compared to the current minimum and maximum for that material and updated if needed. Once all parts were scanned, manually triggering the exit gate stopped the belt, and the LCD displayed the final minimum and maximum values. These values were then used to update the classification thresholds in the main code. The process was repeated for each material to ensure accurate calibration.

Additionally, extensive testing was conducted to develop the fastest, most reliable S-curve for the stepper. Although only plastic parts were meant to remain in the buckets, occasional bouncing caused metal pieces to land in the bins, adding significant weight. To account for this, the stepper was tested

under worst-case conditions using two batteries and two full plastic buckets to ensure it could reliably handle the added load.

Overall, weeks were spent testing and iterating the system to ensure reliable performance. The belt speed was carefully adjusted so that parts didn't slide off when the belt stopped. The exit gate was tilted just enough to ensure parts fell cleanly into the bin after the stepper rotated into position, but not so far forward that they dropped too early. Pre-release timing was also fine-tuned to prevent multiple parts from being released and potentially missing classification.

Additionally, to verify our system's accuracy, we tested various part sequences and recorded the sorting results to confirm each part was classified and directed correctly.

3.6 Limitations and System Tradeoffs

The stepper's S-curve was limited by the need to handle added weight without missing steps or stalling under heavy loads. It required enough initial torque to start moving reliably, which meant a longer delay at the beginning, and the speed could only drop so low before accuracy was affected. Both factors restricted how fast the stepper could operate.

The belt speed was also quite constrained. Plastic parts were slippery and would slide off if the belt was abruptly stopped from too high a speed. Similarly, steel parts had enough inertia to continue moving and fall off the edge if the belt speed was too high.

Finally, a brief delay was added to ensure each part landed in the correct bin before the stepper rotated, slightly reducing system speed but improving sorting accuracy.

4.0 Experience and Recommendations

One area that could be improved is the conveyor system. The belt had low friction, which caused lightweight plastic parts to slide when the belt stopped suddenly. This occasionally led to sorting errors. Using a higher-friction belt material would help keep objects stable during motion and braking, improving overall accuracy.

The stepper motor also had limitations. While it performed well under normal conditions, it struggled when the bins became heavy with metal parts such as steel or aluminum. This affected the sorting speed and risked missed steps. Upgrading to a higher-torque stepper motor would improve reliability and allow the system to operate more quickly without compromising precision.

In the final stages of the project, the lab became crowded, and we often had to switch stations. However, the equipment across stations was not consistent. Some DC motors had different gear ratios, and stepper motors responded differently. As a result, we frequently had to re-tune our belt speed and motion profiles, which slowed progress. Standardizing hardware across all stations would reduce setup time and make debugging and optimization much easier.

Lastly, it would have been very helpful if the course included a brief introduction to Git. Version control is a fundamental skill in collaborative software development. Learning how to manage code versions, track changes, and collaborate within a team using Git would have made our workflow more efficient and prepared us better for future projects.

5.0 Conclusion

This project gave us hands-on experience with embedded systems, sensor integration, and motion control. We successfully built an automated sorting system that met the performance goals, and we gained a stronger understanding of real-time programming, hardware-software interaction, and system debugging. We completed what we set out to achieve and are proud of the system we designed and the work we accomplished.

References

- [1] Y. Shi, "MECH 458/554 Final Project," Department of Mechanical Engineering, University of Victoria, Spring 2025
- [2] OSRAM Opto Semiconductors, *SFH310FA IR Receiver Datasheet*, OSRAM, 2022. [Online]. Available: <https://www.osram.com>.
- [3] TT Electronics, *OPB742 Reflective Sensor Datasheet*, TT Electronics, 2019. [Online]. Available: <https://www.ttelectronics.com>
- [4] TT Electronics, *OPB819Z Slotted Optical Sensor Datasheet*, TT Electronics, 2018. [Online]. Available: <https://www.ttelectronics.com>
- [5] Allegro Microsystems, *Hall-Effect Sensor Selection Guide*, Allegro Microsystems, 2013. [Online]. Available: <https://www.allegromicro.com>
- [6] Banebots, *12V 263RPM Planetary Gearmotor*, BaneBots, 2017. [Online]. Available: <https://banebots.com>.
- [7] Pololu Corporation, *High-Power Motor Driver Carrier 24V23*, Pololu, 2023. [Online]. Available: <https://www.pololu.com/product/1451>
- [8] Soyo Technology, *SY42STH38-0806A Stepper Motor Datasheet*, Soyo Technology, 2022. [Online]. Available: <https://www.soyo.com/stepper-motors>
- [9] STMicroelectronics, *L298 Dual H-Bridge Motor Driver Datasheet*, STMicroelectronics, 2021. [Online]. Available: <https://www.st.com/resource/en/datasheet/l298.pdf>
- [10] Sunfounder, *1602 LCD Module Datasheet*, Sunfounder, 2017. [Online]. Available: <https://www.sunfounder.com/products/1602-lcd-module>
- [11] Atmel Corporation, *ATmega2560 Datasheet*, Atmel Corp., 2014. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-2549-8-bit-AVR-Microcontroller-ATmega640-1280-1281-2560-2561_Datasheet.pdf

Appendix A - Original Statement of Preliminary System Design

The original design used a simple, linear control structure where most of the logic like object detection, ADC reading, classification, and bin movement was handled in the main loop. Interrupts were only planned for flags, and the pause and ramp-down buttons.

As development progressed, the design shifted toward a more interrupt driven approach to improve timing and system reliability. Tasks like ADC sampling, classification, and motor braking were moved into interrupts. This helped the system respond faster and more accurately, especially when sorting objects placed closely together.

One major change was how the ADC was triggered. Originally, we used both rising and falling edge interrupts to bracket the ADC readings. This caused issues when the end sensor triggered too soon. In the final system, we only used a rising edge interrupt and kept reading ADC values while the object was still in range, which simplified the logic and improved accuracy.

Another improvement was braking the DC motor. Originally, braking happened after the bin rotated. We changed this to brake as soon as the exit sensor was triggered, which helped prevent pieces from slipping off the belt.

Overall, many parts of the code were moved from the main loop into interrupts. This made the system flow better and behave more reliably.

Appendix B - Detailed Technical Documentation

Appendix B.1 - Main Project Code

```
#include <stdlib.h>
#include <avr/interrupt.h>
#include <avr/io.h>
#include <util/delay_basic.h>

/* LIBRARIES */
#include "lcd.h"
#include "myutils.h"
#include "StepperProfile.h"
#include "LinkedQueue.h"
```

```

/* GLOBAL VARIABLES */

// For counting objects
volatile int onBeltCount = 0;
volatile int black_count = 0;
volatile int white_count = 0;
volatile int aluminum_count = 0;
volatile int steel_count = 0;
volatile int links = 0;

// ADC values for classification (Reflective Sensor)
volatile unsigned int minValue = 1024; // Initialize to the highest possible 10-bit
value
volatile unsigned int ADC_result;
volatile unsigned int adcValue;

// Link values for queue
link *head; // The ptr to the head of the queue */
link *tail; // The ptr to the tail of the queue */
link *rtnLink = NULL;

// For belt motor
unsigned int forwards = 0b00000111; //CW To match DC motor control signal table?
unsigned int BrakeToVcc = 0b00001111; //Brake to Vcc
unsigned int BrakeToGND = 0b00000011; //Brake to ground
volatile unsigned int motorDir; //Direction of motor

// Assign material to int value
#define WHITE 0
#define STEEL 1
#define BLACK 2
#define ALUMINUM 3

// Stepper position tracking
volatile int currentPos = BLACK;
volatile int curr_step = 0;
char step_table[4] = {0b00110101, 0b00110110, 0b00101110, 0b00101101};

// For flags/logic
volatile int itemAtGate = 0; // set by INT0
volatile int systemPaused = 0; // toggled by INT2
volatile int rampDownRequested = 0; // set by INT3

```

```

/* FUNCTIONS */
void mTimer(int count);
void configurePWM();
void rotateStepperToBin(int fromPos, int toPos);
void rotate_CW_90(void);
void rotate_CCW_90(void);
void rotate_CCW_180(void);
void pauseFunction();
void rampDownFunction();
void RampTimer();

// ----- MAIN -----
int main(void){
    // System clock @ 8 MHz
    CLKPR = 0x80;
    CLKPR = 0x01;

    // Timer1 prescaler set to 8 MHz
    TCCR1B |= _BV(CS11);
    // ----- PORT CONFIGS -----
    DDRF = 0x00;        // Port F to input for RL sensor
    DDRD = 0x00;        // Port D to input for Buttons, EX, and OR
    DDRC = 0xFF;        // Port C to output For LCD
    DDRA = 0x7F;        // Port A to output for Stepper Motor
    DDRB = 0xFF;        // Port B to output for conveyor motor

    cli();// Disables all interrupts
    // ----- ADC SETUP -----
    ADCSRA |= _BV(ADEN);
    ADCSRA |= _BV(ADIE);
    ADMUX |= _BV(REFS0);
    ADCSRA |= _BV(ADPS2) | _BV(ADPS0); // Enable prescale 32x (8MHz -> 250kHz)

    // Set up the Interrupts:
    EIMSK |= (_BV(INT0)); //EX sensor (falling edge)
    EICRA |= _BV(ISC01);

    EIMSK |= (_BV(INT1)); //OR sensor (rising edge)
    EICRA |= _BV(ISC11) | _BV(ISC10);

    EIMSK |= (_BV(INT2)); //Pause Button

```

```

EICRA |= _BV(ISC21) | _BV(ISC20);
EIMSK |= (_BV(INT3)); //Ramp Down Button
EICRA |= _BV(ISC31);

// Enable all interrupts
sei();

// ----- LCD -----
InitLCD(LS_BLINK|LS_ULINE);
LCDClear();

// ----- PWM FOR BELT -----
configurePWM();

// ----- LINKED QUEUE SETUP -----
setup(&head, &tail);

// ----- HOME -----

//Rotate to HE Sensor
while((PINA & 0b10000000) == 0b10000000){

    curr_step--;
    if(curr_step < 0){ //check if at end of cycle
        curr_step = 3; //reset to step 1
    }
    PORTA = step_table[curr_step]; //output current step
    mTimer(15);
}

//Rotate 6 more steps to centerpieces
for(int i = 0; i < 6; i++){
    curr_step--;
    if(curr_step < 0){ //check if at end of cycle
        curr_step = 3; //reset to step 1
    }
    PORTA = step_table[curr_step]; //output current step
    mTimer(15);
}

motorDir = forwards; // Start conveyer belt forward
PORTB = motorDir;

```

```

// ----- MAIN LOOP -----
while(1){

    if(systemPaused == 1){
        pauseFunction();
    }

    // If the exit gate says an item arrived
    if ((itemAtGate == 1)){
        itemAtGate = 0;

        if(!isEmpty(&head)){ //Check to see if there is something in the queue

            int targetBin = head->e.itemCode;

            rotateStepperToBin(currentPos, targetBin);
            currentPos = targetBin;

            if(targetBin == BLACK){
                black_count++;
            }else if(targetBin == WHITE){
                white_count++;
            }else if(targetBin == STEEL){
                steel_count++;
            }else if(targetBin == ALUMINUM){
                aluminum_count++;
            }

            dequeue(&head, &tail, &head);
            onBeltCount--;

        }

        motorDir = forwards;
        PORTB = motorDir;

    }

}

// ----- INTERRUPTS -----

/* 1) End of Travel Sensor - INT0 (Part ready to drop) */

```

```

ISR(INT0_vect){
motorDir = BrakeToVcc;
PORTB = motorDir;
itemAtGate = 1;
}

/* 2) First Optical Sensor - INT1 */
ISR(INT1_vect){
ADCSRA |= _BV(ADSC);
link *newLink = NULL; // A ptr to a link aggregate data type (struct)
initLink(&newLink);
enqueue(&head, &tail, &newLink);
}

/* 3) Pause Button - INT2 */
ISR(INT2_vect){
    mTimer(20);          // Debounce
    if((PIND & 0x04) == 0x04){
        systemPaused = !systemPaused;    // Toggle pause flag

        while((PIND & 0x04) == 0x04);
        mTimer(20);
    }
}

/* 4) Ramp Down Button - INT3 */
ISR(INT3_vect){
    mTimer(20); // Debounce

    if((PIND & 0x08) == 0x00){
        // Clear timer config first

        TCCR3B |= _BV(WGM32);           // WGM32 in TCCR3A
        OCR3A = 30700;                  // Set Output Compare Register A
        TCNT3 = 0;                      // Reset the Timer3 counter to 0
        TCCR3B |= _BV(CS32) | _BV(CS30); // Prescaler = 1024
        TIFR3 |= _BV(OCF3A);            // Clear interrupt flag
        TIMSK3 |= _BV(OCIE3A);           // Enable Compare A interrupt

        while((PIND & 0x08) == 0x00);    // Wait for button release
        mTimer(20);                      // Debounce release
    }
}

```

```

        LCDClear();
        LCDWriteStringXY(0, 0, "Ramp Down");
    }
}

// The interrupt will be triggered if the ADC is done =====
ISR(ADC_vect) {
    ADC_result = ADCL;
    ADC_result += (ADCH << 8);
    adcValue = ADC_result;

    if((PIND & 0b00000010) == 0b00000010){ //ADC continues until part has left

        if (adcValue < minValue) {
            minValue = adcValue;
        }

        ADCSRA |= _BV(ADSC); // Start ADC conversion
        return;

    }else{
        // Classify
        if((minValue > 850) && (minValue < 930)) {
            tail->e.itemCode = WHITE;
            onBeltCount++;
        }
        else if((minValue >= 930)) {
            tail->e.itemCode = BLACK;
            onBeltCount++;
        }
        else if((minValue <= 90)) {
            tail->e.itemCode = ALUMINUM;
            onBeltCount++;
        }
        else if((minValue >= 90) && (minValue <= 850)) {
            tail->e.itemCode = STEEL;
            onBeltCount++;
        }
        else {
            // unknown, just pick something

```

```

        tail->e.itemCode = BLACK;
        onBeltCount++;
    }
    minValue = 1023;
    links++;
}
}

ISR(TIMER3_COMPA_vect) {
    motorDir = BrakeToVcc;
    PORTB = motorDir;

    LCDClear();
    LCDWriteStringXY(0, 0, "B");
    LCDWriteIntXY(1, 0, black_count, 2);
    LCDWriteStringXY(4, 0, "W");
    LCDWriteIntXY(5, 0, white_count, 2);
    LCDWriteStringXY(8, 0, "A");
    LCDWriteIntXY(9, 0, aluminum_count, 2);
    LCDWriteStringXY(12, 0, "S");
    LCDWriteIntXY(13, 0, steel_count, 2);
    LCDWriteStringXY(0,1,"OnBelt:");
    LCDWriteIntXY(7,1, onBeltCount,2);
    LCDWriteIntXY(11,1, links,2);

    while(1){
        PORTA = 0x00;
        PORTB = 0x00;
        cli();
    }
}

// ----- FUNCTIONS -----

void configurePWM(){

    //configure Timer0 for fast PWM (110)
    TCCR0A |= _BV(WGM00); //(sets WGM00 to 1)
    TCCR0A |= _BV(WGM01); //(sets WGM01 to 1)
    TCCR0B &= ~_BV(WGM02); //(sets WGM02 to 0)
    //configure non-inverting PWM mode

```

```

//compare match output will clear on a compare match and set the output compare A to
1) when the timer reaches TOP
TCCR0A &= ~_BV(COM0A0); // (sets COM0A1 to 0)
TCCR0A |= _BV(COM0A1); // (sets COM0A1 to 1)
// clear timer overflow flags
TIFR0 |= _BV(TOV0);
//enable the output compare match interrupt
//set the prescaler to 8
TCCR0B |= _BV(CS00);
TCCR0B |= _BV(CS01);

//set OCR0A to 128 for the 50% duty cycle
OCR0A = 130;
}
void mTimer(int count){
    int i = 0;
    TCCR1B |= _BV(WGM12); //configure Timer1 in CTC modes
    OCR1A = 0x03E8; // set output compare register A to 0x03E8
    TCNT1 = 0x0000; //reset Timer1 counter to 0
    TIFR1 |= _BV(OCF1A); //clear the output compare A match flag
    //wait for the timer to trigger 'count' times
    while(i<count){
        if((TIFR1 & 0x02) == 0x02){ // check if the output compare A match flag is set
            TIFR1 |= _BV(OCF1A); // clear the flag
            i++;
        }
    }
}

void pauseFunction(){

    motorDir = BrakeToVcc;
    PORTB = motorDir;

    while(systemPaused == 1){
        // Show stats
        LCDClear();
        LCDWriteStringXY(0, 0, "B");
        LCDWriteIntXY(1, 0, black_count, 2);
        LCDWriteStringXY(4, 0, "W");
        LCDWriteIntXY(5, 0, white_count, 2);
        LCDWriteStringXY(8, 0, "A");
    }
}

```

```

        LCDWriteIntXY(9, 0, aluminum_count, 2);
        LCDWriteStringXY(12, 0, "S");
        LCDWriteIntXY(13, 0, steel_count, 2);
        LCDWriteStringXY(0,1,"OnBelt:");
        LCDWriteIntXY(7,1, onBeltCount,2);
        LCDWriteIntXY(11,1, links,2);
        mTimer(100);
    }
    LCDClear();
    motorDir = forwards;
    PORTB = motorDir;
}

// ----- STEPPER CONTROL -----
void rotateStepperToBin(int fromPos, int toPos){
    int delta = (toPos - fromPos + 4) % 4; //delta will be 0,1,2,3
    if(delta == 0){
        //in position
        return;
    }else if(delta == 2){
        rotate_CCW_180();

    }else if(delta == 1){
        rotate_CW_90();

    }else if(delta == 3){
        rotate_CCW_90();

    }

}

void rotate_CW_90(){

    for(int i = 0; i < 50; i++){
        curr_step++;
        if(curr_step > 3){ //check if at end of cycle
            curr_step = 0; //reset to step 1

        }

        if(i == 48){ // Two Step Pre-release

```

```

        motorDir = forwards;
        PORTB = motorDir;
    }
    int delayValue = stepper_90[i];
    mTimer(delayValue); // delay after setting conditions
    PORTA = step_table[curr_step]; //output current step

}
}

void rotate_CCW_90(){

    for(int i = 0; i < 50; i++){
        curr_step--;
        if(curr_step < 0){ //check if at end of cycle
            curr_step = 3; //reset to step 1
        }

        if(i == 48){ // Two Step Pre-release
            motorDir = forwards;
            PORTB = motorDir;
        }

        int delayValue = stepper_90[i];
        mTimer(delayValue); // delay after setting conditions
        PORTA = step_table[curr_step]; //output current step
    }

}

void rotate_CCW_180(){

    for(int i = 0; i < 100; i++){
        curr_step--;
        if(curr_step < 0){ //check if at end of cycle
            curr_step = 3; //reset to step 4
        }

        if(i == 98){ // Two Step Pre-release
            motorDir = forwards;
            PORTB = motorDir;
        }
    }
}

```

```

    }

    int delayValue = stepper_180[i];
    mTimer(delayValue); // delay after setting conditions
    PORTA = step_table[curr_step];

}

}

// ----- FIFO FUNCTIONS -----

void setup(link **h, link **t){
    *h = NULL;      /* Point the head to NOTHING (NULL) */
    *t = NULL;      /* Point the tail to NOTHING (NULL) */
    return;
}/*setup*/

void initLink(link **newLink){
    *newLink = malloc(sizeof(link));
    (*newLink)->next = NULL;
    return;
}/*initLink*/

void enqueue(link **h, link **t, link **nL){

    if (*t != NULL){
        (*t)->next = *nL;
        *t = *nL;
    }
    else{
        *h = *nL;
        *t = *nL;
    }
    return;
}/*enqueue*/

void dequeue(link **h, link **t, link **deQueuedLink){

    *deQueuedLink = *h;

```

```

if (*h != NULL) {
    *h = (*h)->next;
}

if(*h == NULL) {
    *t = NULL;
}
return;
}/*dequeue*/

element firstValue(link **h){
return ((*h)->e);
}/*firstValue*/

void clearQueue(link **h, link **t){

link *temp;

while (*h != NULL) {
    temp = *h;
    *h=(*h)->next;
    free(temp);
}

*t = NULL;

return;
}/*clearQueue*/

char isEmpty(link **h){
return(*h == NULL);
}/*isEmpty*/

int size(link **h, link **t){

link    *temp;
int     numElements;

```

```

numElements = 0;
temp = *h;

while(temp != NULL){
    numElements++;
    temp = temp->next;
}

return(numElements);
}/*size*/

```

Appendix B.2 - Calibration File

```

#include <avr/interrupt.h>
#include <avr/io.h>
#include "lcd.h"
#include "myutils.h"

// Define global variables
volatile unsigned int ADC_result;
volatile unsigned int ADC_result_flag;
volatile unsigned int adcConversionCount;

volatile unsigned int i = 0; // Value for debugging

volatile unsigned int minValue;
volatile unsigned int maxValue;
volatile unsigned int adcValue;

volatile unsigned int startFlag = 0; // Flag to control ADC loop

unsigned int forwards = 0b00001011;
unsigned int backwards = 0b00000111;
unsigned int BrakeToVcc = 0b00001111;
unsigned int BrakeToGND = 0b00000011;

volatile unsigned int state; // Direction of motor

void configurePWM();
void mTimer(int count);

```

```

int main(void) {
    CLKPR = 0x80;           // Restrict MCU clock to 8 MHz
    CLKPR = 0x01;           // Set timer to 1 MHz
    TCCR1B |= _BV(CS11);    // Prescaler set to 8

    // Configure Ports
    DDRF = 0x00;            // Port F: input for RL sensor
    DDRD = 0x00;            // Port D: input for buttons
    DDRC = 0xFF;            // Port C: output for LCD
    DDRA = 0xFF;            // Port A: output for stepper motor
    DDRB = 0xFF;            // Port B: output for conveyor motor

    cli();                  // Disable interrupts

    // External interrupts
    EIMSK |= _BV(INT1);     // Enable INT1
    EICRA |= _BV(ISC10);    // INT1: any edge
    EIMSK |= _BV(INT0);     // Enable INT0
    EICRA |= _BV(ISC01) | _BV(ISC00); // INT0: rising edge

    // Configure ADC
    ADCSRA |= _BV(ADEN);    // Enable ADC
    ADCSRA |= _BV(ADIE);    // Enable ADC interrupt
    ADMUX |= _BV(REFS0);    // AVcc as reference

    sei();                  // Enable interrupts

    ADCSRA |= _BV(ADSC);    // Initial conversion

    // LCD setup
    InitLCD(LS_BLINK | LS_ULINE);
    LCDClear();

    // PWM setup
    configurePWM();

    state = backwards;
    PORTB = state;

    adcConversionCount = 0;
    minValue = 9999;

```

```

while (1) {
    if (startFlag) {
        while (startFlag) {
            if (ADC_result_flag) {
                adcValue = ADC_result;

                if (adcValue < minValue) {
                    minValue = adcValue;
                    i++;
                }

                ADC_result_flag = 0x00;
                ADCSRA |= _BV(ADSC); // Start new conversion
            }
        }
    }
}

return 0;
}

// INT0 - End of Travel Sensor
ISR(INT0_vect) {
    state = BrakeToVcc;
    PORTB = state;

    LCDClear();
    LCDWriteStringXY(0, 0, "Min:");
    LCDWriteIntXY(5, 0, minValue, 4);
    LCDWriteStringXY(0, 1, "i: ");
    LCDWriteIntXY(10, 1, i, 4);
}

// INT1 - First Optical Sensor
ISR(INT1_vect) {
    startFlag = !startFlag; // Toggle flag
}

// ADC Conversion Complete
ISR(ADC_vect) {
    ADC_result = ADCL;

```

```

    ADC_result += (ADCH << 8);
    ADC_result_flag = 1;
    adcConversionCount++;
    LCDWriteIntXY(5, 0, ADC_result, 4);
}

void configurePWM() {
    // Timer0 Fast PWM
    TCCR0A |= _BV(WGM00);
    TCCR0A |= _BV(WGM01);
    TCCR0B &= ~_BV(WGM02);

    // Non-inverting mode
    TCCR0A &= ~_BV(COM0A0);
    TCCR0A |= _BV(COM0A1);

    TIFR0 |= _BV(TOV0);    // Clear overflow flag

    // Prescaler = 64
    TCCR0B |= _BV(CS00);
    TCCR0B |= _BV(CS01);

    DDRA = 0xFF;
    DDRB = 0xFF;

    OCR0A = 60;
    DDRB |= (1 << PB7);
}

void mTimer(int count) {
    int i = 0;
    TCCR1B |= _BV(WGM12);    // CTC mode
    OCR1A = 0x03E8;
    TCNT1 = 0x0000;
    TIFR1 |= _BV(OCF1A);

    while (i < count) {
        if ((TIFR1 & 0x02) == 0x02) {
            TIFR1 |= _BV(OCF1A);
            i++;
        }
    }
}

```

```
}
```

Appendix B.3 - Stepper Library

```
#ifndef STEPPERPROFILE_H_
#define STEPPERPROFILE_H_

char stepper_90[50] = {
    70, 19.9761033520416, 19.7955443225146, 18.7052681744403,
    16.138556842889, 13.5485007529216, 11.5559365654134, 10.0838339000431,
    9.00132840220764, 8.20572680315635, 7.62102278842843, 7.19131450846789,
    6.87551528331582, 6.64342959306149, 6.47286626401765, 6.34751666079763,
    6.25539531728365, 6.18769393081955, 6.13793914485662, 6.1013735905082,
    6.07450100450606, 6.05475192941856, 6.04023803162025, 6.02957154580426,
    6.02173258198876, 6.01597160740342, 6.01597160740342, 6.02173258198876,
    6.02957154580426, 6.04023803162025, 6.05475192941856, 6.07450100450606,
    6.1013735905082, 6.13793914485662, 6.18769393081955, 6.25539531728365,
    6.34751666079763, 6.47286626401765, 6.64342959306149, 6.87551528331582,
    7.19131450846789, 7.62102278842843, 8.20572680315635, 9.00132840220764,
    10.0838339000431, 11.5559365654134, 13.5485007529216, 16.138556842889,
    18.7052681744403, 19.7955443225146
};

char stepper_180[100] = {
    70, 19.9761033520416, 19.7955443225146, 18.7052681744403,
    16.138556842889, 13.5485007529216, 11.5559365654134, 10.0838339000431,
    9.00132840220764, 8.20572680315635, 7.62102278842843, 7.19131450846789,
    6.87551528331582, 6.64342959306149, 6.47286626401765, 6.34751666079763,
    6.25539531728365, 6.18769393081955, 6.13793914485662, 6.1013735905082,
    6.07450100450606, 6.05475192941856, 6.04023803162025, 6.02957154580426,
    6.02173258198876, 6.01597160740342, 6.011737778934, 6.008626273538,
    6.00633958056042, 6.0046590548636, 6.00342401078671, 6.00251635797618,
    6.00184931002229, 6.00135908626313, 6.00099881331327, 6.00073404320375,
    6.00053945959452, 6.00039645711946, 6.00029136241002, 6.00021412669822,
    6.00015736499051, 6.00011564994205, 6.00008499291394, 6.00006246259438,
    6.00004590471741, 6.00003373607999, 6.00002479316196, 6.0000182208745,
    6.00001339079978, 6.00000984110388, 6.00000723237799, 6.00000723237799,
    6.00000984110388, 6.00001339079978, 6.0000182208745, 6.00002479316196,
    6.00003373607999, 6.00004590471741, 6.00006246259438, 6.00008499291394,
    6.00011564994205, 6.00015736499051, 6.00021412669822, 6.00029136241002,
    6.00039645711946, 6.00053945959452, 6.00073404320375, 6.00099881331327,
```

```

6.00135908626313, 6.00184931002229, 6.00251635797618, 6.00342401078671,
6.0046590548636, 6.00633958056042, 6.008626273538, 6.011737778934,
6.01597160740342, 6.02173258198876, 6.02957154580426, 6.04023803162025,
6.05475192941856, 6.07450100450606, 6.1013735905082, 6.13793914485662,
6.18769393081955, 6.25539531728365, 6.34751666079763, 6.47286626401765,
6.64342959306149, 6.87551528331582, 7.19131450846789, 7.62102278842843,
8.20572680315635, 9.00132840220764, 10.0838339000431, 11.5559365654134,
13.5485007529216, 16.138556842889, 18.7052681744403, 19.7955443225146
};

#endif /* STEPPERPROFILE_H_ */

```

Appendix B.4 - Circuit Diagram

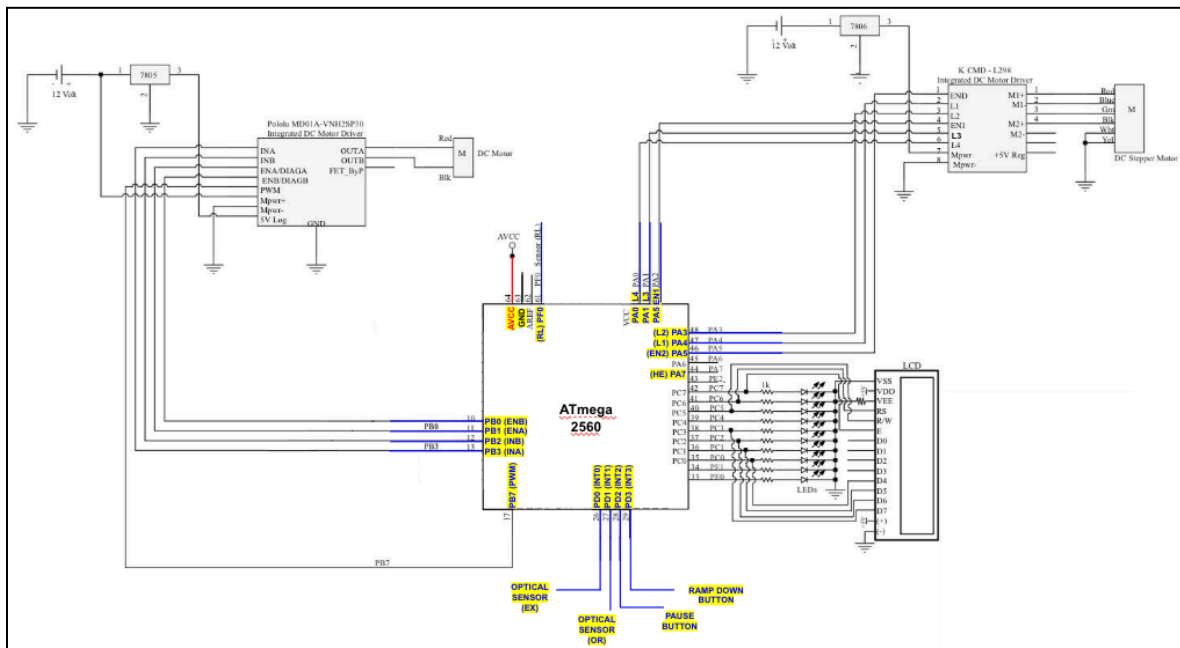


Figure B.1: Circuit diagram

Appendix B.5 - Block Diagram

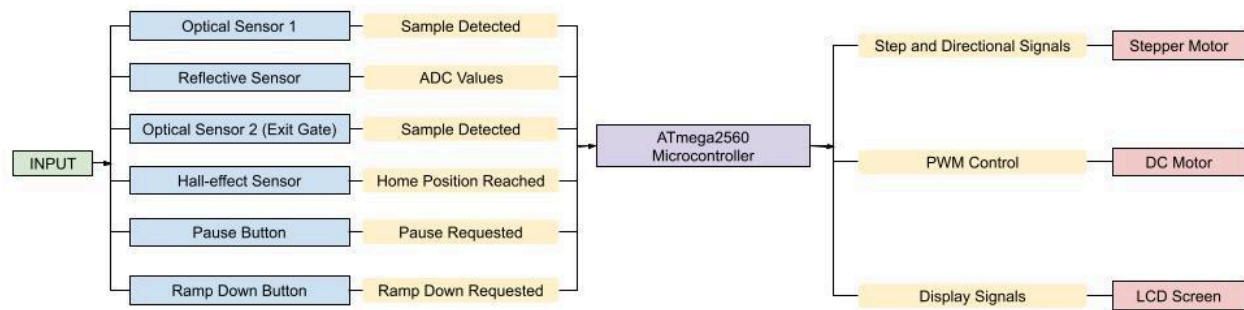


Figure B.2: Block Diagram